lecture_27

October 9, 2026

1 Lecture 27: Genetic Algorithm - Motivation & Pseudocode

```
{note} Simulated Annealing (Lectures 24-26) refines a single solution, one
neighbourhood move at a time - cheap per iteration, but with only one "thread"
of search, it depends entirely on its cooling schedule to avoid getting stuck.
This lecture introduces **population search**, Lecture 23's second paradigm:
instead of one solution, maintain a whole population, and let solutions combine
and compete with each other. The **Genetic Algorithm (GA)**, this module's
example, is developed here as pseudocode; Lecture 28 specializes it for the
Ackley function.
```

Learning Objectives

By the end of this notebook, you will be able to: 1. Explain why maintaining a population of solutions, rather than one, changes how a search explores and exploits a solution landscape. 2. Read and interpret the Genetic Algorithm's pseudocode, including selection, crossover, mutation, and natural selection. 3. Hand-trace one generation of a Genetic Algorithm given a small population and its genetic operators.

Prerequisites: Simulated Annealing - Benchmarking (Lecture 26); Metaheuristics (Lecture 23).

Estimated time: 50 minutes

1.1 Why Population Search?

Simulated Annealing carries exactly one current solution forward at every iteration. Its only defense against getting trapped near a local optimum is the temperature schedule's willingness to accept a worse move — and Lecture 26 showed just how sensitive that defense is to getting the schedule right. **Population search** takes a different approach entirely: instead of one solution taking a random walk through the landscape, a whole *population* of solutions is maintained and evolved together, inspired by biological natural selection.

The **Genetic Algorithm (GA)** is the best-known population search method. Each generation, it: 1. **Selects** promising parent solutions from the current population (favouring, but not exclusively choosing, fitter solutions), 2. **Recombines** pairs (or larger groups) of parents through **crossover**, producing offspring that inherit traits from each parent, 3. **Mutates** offspring with a small probability, introducing traits not present in either parent, 4. **Replaces** the population with a new one drawn from the parents and offspring together, favouring fitness.

{note} Crossover is what fundamentally distinguishes population search from local search: a neighbourhood move (Lecture 24) perturbs *one* solution, while crossover combines traits from *two or more* solutions that may be exploring very different regions of the landscape. This is also why a population, even one that hasn't converged, is inherently more resistant to getting trapped: different members can be exploring different local optima at the same time, and crossover occasionally combines the best traits of each.

The tradeoff is cost: evaluating and evolving an entire population every generation is more expensive per iteration than Simulated Annealing's single-solution move. Lecture 33's comparison will make this cost concrete.

1.2 Notation

Symbol	Meaning
s	The current population of solutions
s	The best solution in the current population (local best)
s^*	The best solution found across all generations (global best)
$f(s)$	Objective function value (to be minimized) at solution s
SM, μ	Selection method, and number of parents it selects
CM, λ, ρ	Crossover method; number of offspring produced; number of parents combined per offspring
MM, ϵ	Mutation method, and the probability an offspring is mutated
NM, κ	Natural selection (replacement) method, producing the next generation's population; diversity parameter

This notation deliberately leaves SM, CM, MM, and NM unspecified — they are **pluggable operators**. Lecture 28 will instantiate them for real-valued vectors (roulette-wheel selection, arithmetic crossover, Gaussian mutation); Lecture 29 will instantiate different ones for permutations (tournament selection, order crossover, swap mutation), once it asks whether a calibration carries over to a routing problem. The loop below — the *engine* — stays the same either way.

1.3 Pseudocode

1. **Procedure** GA$(s_o, (\text{SM}, \mu), (\text{CM}, \lambda, \rho), (\text{MM}, \epsilon), (\text{NM}, \kappa))$
2. $s \leftarrow s_o$ // initialise the population
3. $s \leftarrow \text{argmin}\{f(s) : s \in s\}$ // local best in the initial population
4. $s^* \leftarrow s$ // global best so far
5. **while** !converged **do**

6. $s_p \leftarrow \text{SM}(s, \mu)$ // select μ parents from the population
7. $s_c \leftarrow \text{CM}(s_p, \lambda, \rho)$ // generate λ offspring, ρ parents per offspring
8. $s_c \leftarrow \text{MM}(s_c, \epsilon)$ // mutate offspring with probability ϵ
9. $s \leftarrow \text{NM}(s \cup s_c, \kappa)$ // replace the population from parents + offspring
10. $s \leftarrow \text{argmin}\{f(s) : s \in s\}$ // local best in the new population
11. **if** $f(s) < f(s^*)$ **then**
12. $s^* \leftarrow s$
13. **end if**
14. **end while**
15. **return** s^*

`{tip} Notice line 9 replaces the population from $\boldsymbol{s} \cup \boldsymbol{s_c}$ - the *parents* as well as the *offspring* - rather than discarding the parents outright. This is what Lecture 28's implementation will call **elitist replacement**: it guarantees the population's best solution never gets worse from one generation to the next, since a strong parent can always survive into the next generation if none of its offspring beat it.`

1.4 Hand-Traced Example

Represent each solution as a 4-bit string, with objective $f(s) = $ number of 0s in s (so the global optimum is $s^* = 1111$, $f = 0$). Start from a population of three:

Solution	Bits	f (zeros)
s_1	1101	1
s_2	0110	2
s_3	0001	3

Trace one generation with: SM selects the $\mu = 2$ fittest solutions; CM performs single-point crossover at position 2 (after the first two bits) on $\rho = 2$ parents, producing $\lambda = 1$ offspring; MM flips the last bit with probability ϵ (given as "mutate" below); NM keeps the 3 fittest solutions from parents + offspring combined.

Step	Operation	Result
Selection (SM)	Pick the $\mu = 2$ fittest of $\{s_1, s_2, s_3\}$	Parents: $s_1 =$ `1101` ($f = 1$), $s_2 =$ `0110` ($f = 2$)
Crossover (CM)	Single-point at position 2: first 2 bits of s_1 + last 2 bits of s_2	Offspring $c =$ `11`+`10` = `1110` ($f = 1$)
Mutation (MM)	Flip the last bit of c	$c =$ `1111` ($f = 0$)
Natural Selection (NM)	Keep 3 fittest of $\{s_1(1), s_2(2), s_3(3), c(0)\}$	New population: `1111` ($f = 0$), `1101` ($f = 1$), `0110` ($f = 2$) — s_3 (`0001`, $f = 3$) is dropped

In a single generation, crossover combined s_1's strong first half with s_2's (weaker) second half, and mutation then happened to fix the one remaining zero — landing exactly on the global optimum

$s^* = 1111$ ($f = 0$). This is, admittedly, a lucky mutation; in general GA relies on many generations of accumulated improvement, not one fortunate flip. What the trace does demonstrate reliably is **how crossover creates a new solution genuinely different from either parent, and how elitist replacement never lets the population's best solution get worse.**

1.5 In-Class Exercise

1.5.1 Exercise 1 — A Generation Without a Lucky Mutation

Repeat the trace above from the same starting population, but with the mutation step disabled ($\epsilon = 0$, so c stays as `1110`). What is the new population after natural selection, and what is $f(s^*)$ after this generation? Confirm that s^* still strictly improves compared to the initial population's best (s_1, $f = 1$), even without a mutation.

Step	Operation	Result
Selection	Same as above	Parents: `1101` ($f = 1$), `0110` ($f = 2$)
Crossover	Same as above	Offspring $c =$ `1110` ($f = 1$)
Mutation	Disabled	$c =$ `1110` ($f = 1$)
Natural Selection	Keep 3 fittest of {`1101`(1), `0110`(2), `0001`(3), `1110`(1)}	New population: `1101` (1), `1110` (1), `0110` (2) — tie broken arbitrarily between the two $f = 1$ solutions

Even without the lucky mutation, the population's best value stays at $f = 1$ (no worse than before) — elitist replacement guarantees this — though it does not improve either, since crossover alone did not produce a strictly better offspring this generation. Reaching $f = 0$ would need either a helpful mutation, as in the main example, or another generation of crossover among the current population's diversity.

1.6 Take-Away Exercises

1.6.1 Exercise 1 — A Larger Population

Extend this lecture's example to a population of five 4-bit strings of your choosing (include at least one string with $f = 0$ or $f = 1$ already, and at least one with $f = 3$ or $f = 4$, to keep some diversity). Trace one generation with $\mu = 3$ parents, $\lambda = 2$ offspring (one crossover pair each), and your own choice of crossover points. Does natural selection ever discard a solution that turns out, in hindsight, to have been useful?

1.6.2 Exercise 2 — Crossover Point Sensitivity

Using this lecture's original population and parents ($s_1 =$`1101`, $s_2 =$`0110`), redo the crossover step with the split at position 1 (after the first bit) and again at position 3 (after the first three bits), instead of position 2. Compare the three resulting offspring (before mutation) and their f values.

Does the choice of crossover point matter here, and would you expect it to matter more or less on a longer bit string?

1.7 Circling Back

- **Lecture 23 (Metaheuristics)**: the Genetic Algorithm is this module's example of the *population search* paradigm — Lecture 23's comparison table listed diversity (many candidates explored at once) as population search's chief defense against premature convergence, exactly what this lecture's crossover step demonstrated.
- **Lecture 24 (Simulated Annealing: Motivation & Pseudocode)**: contrast the two pseudocodes directly — SA's single `while` loop moves one solution through a neighbourhood; GA's loop moves an entire population through selection, crossover, mutation, and replacement. Both still track a "best so far" (s^*) and both still stop on a convergence criterion.

1.8 Moving Forward

- **Lecture 28 (Genetic Algorithm: Algorithm)**: implements this pseudocode in Python, specializing SM, CM, MM, and NM for real-valued vectors, and applies it to the Ackley function Lecture 25 used for Simulated Annealing.

1.9 Further Reading

- Holland, J.H. (1975). *Adaptation in Natural and Artificial Systems.* University of Michigan Press — the foundational text introducing genetic algorithms.
- Goldberg, D.E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning.* Addison-Wesley — the standard applied reference, including selection and crossover operator design.
- Mitchell, M. (1998). *An Introduction to Genetic Algorithms.* MIT Press — an accessible, example-driven introduction.